

Exception Handling, Testing & Debugging in Java and Python

COURSE OBJECTIVES

- Exception handling, assertions, user-defined exceptions, input validation
- Unit testing frameworks: **JUnit 5** (Java), **pytest** (Python)
- Writing test suites and following a **TDD** (Test-Driven Development) approach
- Debugging concurrent and distributed applications

🎯 **Goal:** Write robust, testable, and debuggable code in Java and Python.

```
// Java Exception Handling  
  
try {  
    executeCode();  
} catch (Exception e) {  
    log(e);  
}
```

```
# Python pytest Framework  
  
def test_robustness():  
    with pytest.raises(Error):  
        execute_code()  
    assert status == True
```

Java Exception Handling: try-catch-finally

Java

💡 Concepts

- **Hierarchy:** Throwable → Exception / Error
- **Checked vs Unchecked:** All inherit from Exception, RuntimeExceptions are unchecked
- **try** block for risky code
- **catch** block for handling
- **finally** block always executes for cleanup

☕ BasicExceptionDemo.java

```
public class BasicExceptionDemo {  
    public static void divide(int a, int b) {  
        try {  
            int result = a / b;  
            System.out.println("Result: " + result);  
        } catch (ArithmeticException e) {  
            System.err.println("Error: Division by zero!");  
            System.err.println("Message: " + e.getMessage());  
        } finally {  
            System.out.println("Cleanup executed");  
        }  
    }  
  
    public static void main(String[] args) {  
        divide(10, 0);  
        divide(10, 2);  
    }  
}
```

Python Exception Handling: try-except-else-finally

Python

💡 Concepts

- **Hierarchy:** All built-in exceptions inherit from `BaseException` → `Exception`
- `try` block contains risky code
- `except` blocks handle specific errors
- `else` block runs **only** if no exception occurs
- `finally` block always executes for cleanup

exception_demo.py

```
def divide(a, b):
    try:
        result = a / b
    print(f"Result: {result}")
    except ZeroDivisionError as e:
        print("Error: Division by zero!")
        print(f"Message: {e}")
    except TypeError as e:
        print(f"Error: Invalid type: {e}")
    else:
        print("No exception occurred")
    finally:
        print("Cleanup executed\n")

# Testing the function
divide(10, 0) # Triggers ZeroDivisionError
divide(10, 2) # Triggers else block
divide(10, "2") # Triggers TypeError
```

Handling Multiple Exceptions

Java

 ExceptionProcessor.java

```
public static void processArray(String[] arr, int index) {
    try {
        String value = arr[index];           // AI00BE
        int number = Integer.parseInt(value); // NFE
        int result = 100 / number;           // AE
        System.out.println("Result: " + result);
    } catch (ArrayIndexOutOfBoundsException |
            NumberFormatException e) {
        System.err.println("Invalid input: " + e.getMessage());
    } catch (ArithmeticException e) {
        System.err.println("Math error: " + e.getMessage());
    } catch (Exception e) {
        System.err.println("Unexpected error: " + e.getMessage());
    }
}
```

Python

 exception_processor.py

```
def process_array(arr, index):
    try:
        value = arr[index]
        number = int(value)
        result = 100 / number
    print(f"Result: {result}")
    except (IndexError, ValueError) as e:
        print(f"Invalid input: {e}")
    except ZeroDivisionError as e:
        print(f"Math error: {e}")
    except Exception as e:
        print(f"Unexpected error: {e}")
```

Custom Exceptions in Java

💡 **Domain-specific error types** improve clarity ✔️ **Checked** (extends `Exception`) vs **unchecked** (extends `RuntimeException`)

Java

🔥 Definition (Custom Types)

```
class InsufficientFundsException extends Exception {
    private final double shortage;
    public InsufficientFundsException(double shortage) {
        super("Insufficient funds. Short by: " + shortage);
        this.shortage = shortage;
    }
    public double getShortage() { return shortage; }
}

class InvalidAccountException extends RuntimeException {
    public InvalidAccountException(String accountNumber) {
        super("Invalid account number: " + accountNumber);
    }
}
```

Java

🔥 Usage (BankAccount.java)

```
class BankAccount {
    private final String accountNumber;
    private double balance;

    public BankAccount(String accNum, double initialBalance) {
        if (accNum == null || accNum.length() != 10) {
            throw new InvalidAccountException(accNum);
        }
        this.accountNumber = accNum;
        this.balance = initialBalance;
    }

    public void withdraw(double amount) throws InsufficientFundsException {
        if (amount > balance)
            throw new InsufficientFundsException(amount - balance);
        balance -= amount;
    }
}
```

Custom Exceptions in Python

- 💡 Inherit from **Exception** or a more specific base
- ✓ Add context (e.g., amount, balance) to the exception

Python

Definition (Custom Types)

```
class InsufficientFundsException(Exception):
    def __init__(self, amount, balance):
        self.amount = amount
self.balance = balance
self.shortage = amount - balance
super().__init__(
    f"Insufficient funds. Required: {amount}, Available:
{balance}"
)

class InvalidAccountException(ValueError):
    def __init__(self, account_number):
        super().__init__(f"Invalid account number:
{account_number}")
```

Python

Usage (bank_account.py)

```
class BankAccount:
    def __init__(self, account_number, initial_balance):
        if not account_number or len(account_number) != 10:
            raise InvalidAccountException(account_number)
        self.account_number = account_number
self.balance = initial_balance
def withdraw(self, amount):
    if amount > self.balance:
        raise InsufficientFundsException(amount,
self.balance)
    self.balance -= amount
```

Assertions for Invariants (Java)

Java

☰ Concepts

- Enable with `-ea` (enable assertions) flag in JVM
- Use for internal checks and invariants, **not** for user input validation

i When to use

- **Preconditions & postconditions:** ensure valid state transitions.
- **Invariants:** conditions that must always hold true during development.
- **Production:** typically disabled to avoid performance overhead.

Java

🔥 AssertionDemo.java

```
public static double calculateSquareRoot(double number) {
    assert number >= 0 : "Number must be non-negative, got: " + number;
    return Math.sqrt(number);
}

public static int[] sortArray(int[] arr) {
    int[] sorted = arr.clone();
    java.util.Arrays.sort(sorted);

    // Verify postcondition: array is actually sorted
    for (int i = 0; i < sorted.length - 1; i++) {
        assert sorted[i] <= sorted[i + 1] : "Array not properly sorted";
    }

    return sorted;
}
```

Assertions for Invariants (Python)

Python

≡ Concepts

- Syntax: `assert condition, "message"`
- Turned off completely by running with the `python -O` flag

i When to use

- **Preconditions & postconditions:** ensure valid inputs and outputs for functions.
- **Class invariants:** maintain valid state across method calls (e.g., data length).
- Never use for **business logic** or expected user input validation.

Python

assertions_demo.py

```
def calculate_square_root(number):
    assert number >= 0, f"Number must be non-negative, got: {number}"
    return number ** 0.5
def sort_array(arr):
    sorted_arr = sorted(arr)
    for i in range(len(sorted_arr) - 1):
        assert sorted_arr[i] <= sorted_arr[i + 1], "Array not sorted"
    return sorted_arr
class Stack:
    def __init__(self, capacity):
        self.data = []
        self.capacity = capacity
    def push(self, value):
        # Class invariant check
        assert len(self.data) < self.capacity, "Stack overflow"
        self.data.append(value)
        assert 0 <= len(self.data) <= self.capacity
```


Input Validation (Java & Python)

💡 **Key ideas:** ✓ Validate at system boundaries ✓ Throw/raise specific exceptions with clear messages

Java

🔥 Validator.java

```
public static String validateEmail(String email) {
    if (email == null || email.trim().isEmpty())
        throw new IllegalArgumentException("Email cannot be empty");

    String regex = "^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2}$";
    if (!email.matches(regex))
        throw new IllegalArgumentException("Invalid email: " + email);

    return email.trim().toLowerCase();
}
```

Python

🐍 validator.py

```
import re
class ValidationError(Exception):
    pass
def validate_email(email: str) -> str:
    if not email or not email.strip():
        raise ValidationError("Email cannot be empty")

    regex = r"^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2}$"
    if not re.match(regex, email):
        raise ValidationError(f"Invalid email: {email}")

    return email.strip().lower()
```

Unit Testing in Java with JUnit 5

Java

Concepts

- **Core annotations:** Define test lifecycle and execution
- `@Test` marks a method as a test case
- `@BeforeEach`, `@AfterEach` for setup and teardown per test
- `@BeforeAll` for one-time setup
- `@ParameterizedTest` for data-driven tests
- Use `assertEquals` and `assertThrows` for validation

 CalculatorTest.java

```
class Calculator {
    int add(int a, int b) { return a + b; }
    int divide(int a, int b) {
        if (b == 0) throw new ArithmeticException("Division by zero");
        return a / b;
    }
}

import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {
    private Calculator calc;

    @BeforeEach
    void init() {
        calc = new Calculator();
    }

    @Test
    void add_positive() {
        assertEquals(5, calc.add(2, 3));
    }

    @Test
    void divide_by_zero_throws() {
        assertThrows(ArithmeticException.class, () -> calc.divide(10, 0));
    }
}
```

Unit Testing in Python with pytest

Python

💡 Concepts

- **No base class needed:** Write tests as simple functions
- Use `fixtures` for shared setup and teardown logic
- Use `pytest.raises` to elegantly test for expected exceptions

calculator.py & test_calculator.py

```
# calculator.py
class Calculator:
    def add(self, a, b):
        return a + b
    def divide(self, a, b):
        if b == 0:
            raise ZeroDivisionError("Division by zero")
        return a / b

# test_calculator.py
import pytest
from calculator import Calculator
@pytest.fixture
def calc():
    return Calculator()

def test_add_positive(calc):
    assert calc.add(2, 3) == 5
def test_divide_by_zero(calc):
    with pytest.raises(ZeroDivisionError, match="Division by zero"):
        calc.divide(10, 0)
```

TDD: Red-Green-Refactor



Steps:



Red: Write a failing test



Green: Write minimal code to make it pass



Refactor: Clean up code, keep tests green

Java

StringUtilsTest.java & StringUtils.java

```
// test
class StringUtilsTest {
    @Test
    void reverse_basic() {
        StringUtils utils = new StringUtils();
        assertEquals("cba", utils.reverse("abc"));
    }
    @Test
    void reverse_null_throws() {
        StringUtils utils = new StringUtils();
        assertThrows(IllegalArgumentException.class,
            () -> utils.reverse(null));
    }
}

// implementation
class StringUtils {
    String reverse(String input) {
        if (input == null)
            throw new IllegalArgumentException("Input null");
        return new StringBuilder(input).reverse().toString();
    }
}
```

Python

test_string_utils.py & string_utils.py

```
# test
class TestStringUtils:
    def test_reverse_basic(self):
        utils = StringUtils()
        assert utils.reverse("abc") == "cba"
    def test_reverse_none(self):
        utils = StringUtils()
        with pytest.raises(ValueError):
            utils.reverse(None)

# implementation
class StringUtils:
    def reverse(self, s):
        if s is None:
            raise ValueError("Input cannot be None")
        return s[::-1]
```

Mocking with Mockito (Java)

- 💡 Use mocks to **isolate code** from external systems
- ✅ **Verify interactions** between objects

Java

🔥 Types & Service

```
interface PaymentGateway {
    boolean processPayment(String card, double amount);
    String getTransactionId();
}

class OrderService {
    private final PaymentGateway gateway;

    public OrderService(PaymentGateway gateway) {
        this.gateway = gateway;
    }

    public String placeOrder(String card, double amount) {
        if (gateway.processPayment(card, amount)) {
            return "Order placed: " + gateway.getTransactionId();
        }
        return "Payment failed";
    }
}
```

Java

🔥 Test with Mockito

```
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class OrderServiceTest {
    @Test
    void successful_order() {
        // Arrange
        PaymentGateway gateway = mock(PaymentGateway.class);
        when(gateway.processPayment("1234", 100.0)).thenReturn(true);
        when(gateway.getTransactionId()).thenReturn("TXN-1");

        OrderService service = new OrderService(gateway);

        // Act
        String result = service.placeOrder("1234", 100.0);

        // Assert
        assertEquals("Order placed: TXN-1", result);
        verify(gateway).processPayment("1234", 100.0);
        verify(gateway).getTransactionId();
    }
}
```

Mocking with unittest.mock (Python)

Python

💡 Concepts

- Use `Mock`, `MagicMock`, or `patch` to replace real dependencies.
- Isolate code under test from external systems (like databases, APIs).
- Configure mock behavior using `return_value` or `side_effect`.
- Verify interactions with `assert_called_once()` and similar assertions.

```
test_order_service.py

from unittest.mock import Mock
import pytest

class PaymentGateway:
    def process_payment(self, card, amount): ...
    def get_transaction_id(self): ...

class OrderService:
    def __init__(self, gateway):
        self.gateway = gateway
    def place_order(self, card, amount):
        if self.gateway.process_payment(card, amount):
            return f"Order placed: {self.gateway.get_transaction_id()}"
        return "Payment failed"

class TestOrderService:
    def test_successful_order(self):
        gateway = Mock(spec=PaymentGateway)
        gateway.process_payment.return_value = True
        gateway.get_transaction_id.return_value = "TXN-1"
        service = OrderService(gateway)
        result = service.place_order("1234", 100.0)

        assert result == "Order placed: TXN-1"
        gateway.process_payment.assert_called_once_with("1234", 100.0)
        gateway.get_transaction_id.assert_called_once()
```

Test Suites & Structure

💡 **Best practices:** ✓ Separate unit / integration / e2e ✓ Shared fixtures in conftest.py ✓ Use markers and select tests by type

Java

ServiceTestSuite.java

```
import org.junit.platform.suite.api.*;

@Suite
@SelectClasses({
    UserServiceTest.class,
    OrderServiceTest.class
})
class ServiceTestSuite {
    // The class remains empty,
    // used only as a holder for the above annotations
}
```

Python

pytest usage

```
# Layout:
tests/unit/test_*.py -> unit tests
tests/integration/test_*.py -> integration
# Markers in code:
@pytest.mark.unit
def test_basic(): ...

@pytest.mark.integration
def test_db(): ...

# CLI Commands:
$ pytest tests/unit
$ pytest -m "integration and not slow"
```

Thread Safety & Race Conditions (Java)

Debugging tips:

🔍 Look for **unsynchronized shared state**

🔥 Use thread dumps (jstack)

🔒 Use **synchronized** or **ReentrantLock**

Java

🔥 Problem (Race Condition)

```
class BankAccount {
    private int balance = 1000;

    public void withdraw(int amount) {
        if (balance >= amount) {
            // context switch can occur here
            balance -= amount;
        }
    }
}
```

Java

🔥 Fix with Synchronization

```
class SafeBankAccount {
    private int balance = 1000;

    public synchronized void withdraw(int amount) {
        if (balance >= amount) {
            balance -= amount;
        }
    }
}
```


Thread Safety & Locks (Python)

Debugging tools:

 threading.enumerate()

 faulthandler.dump_traceback()
(stuck threads)

 asyncio:loop.set_debug(True)

Python

 Problem (Race Condition)

```
import threading, time

class BankAccount:
    def __init__(self):
        self.balance = 1000
    def withdraw(self, amount):
        if self.balance >= amount:
            # Context switch can occur here
            time.sleep(0.001)
            self.balance -= amount
```

Python

 Fix with Lock

```
class SafeBankAccount:
    def __init__(self):
        self.balance = 1000
    self.lock = threading.Lock()

    def withdraw(self, amount):
        with self.lock:
            if self.balance >= amount:
                time.sleep(0.001)
                self.balance -= amount
```

Patterns for Distributed Debugging

Python

☰ Core problems

- Partial failures, timeouts, retries
- Hard to trace a request across distributed services and microservices

💡 Patterns & Tools

- **Correlation IDs:** Pass unique IDs through requests and log them.
- **Resilience:** Circuit breaker and retry with exponential backoff.
- **Logging:** Use structured logging (JSON) for easy querying.
- **Observability:** Health checks, metrics, tracing tools (Jaeger/Zipkin).

Python

retry.py

```
import time, logging
from functools import wraps
def retry_with_backoff(max_retries=3, base_delay=1):
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            for attempt in range(max_retries):
                try:
                    return fn(*args, **kwargs)
                except Exception as e:
                    if attempt == max_retries - 1:
                        raise
                    wait = base_delay * (2 ** attempt)
                    logging.warning(f"Retry {attempt+1}...")
                    time.sleep(wait)
            return wrapper
        return decorator
```

Logging for Debugging

📖 Guidelines:

✅ Use appropriate levels (DEBUG, INFO, WARN, ERROR)

✅ Never log secrets (passwords, tokens)

✅ Prefer structured logs (JSON) in distributed systems

Java

🔥 MyService.java (SLF4J)

```
private static final org.slf4j.Logger log =
    org.slf4j.LoggerFactory.getLogger(MyService.class);

public void process(String userId) {
    log.info("Processing request for user {}", userId);
    try {
        // ... business logic ...
    } catch (Exception e) {
        log.error("Failed to process request for {}", userId, e);
    }
}
```

Python

🔗 service.py (logging)

```
import logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def handle_request(user_id):
    logger.info("Processing request for user %s", user_id)
    try:
        # ... business logic ...
    pass
except Exception:
    logger.exception("Failed to process request")
```

Summary & Best Practices

Exception Handling

- Use **specific exception types** for clarity
- Clean up resources using `finally`, try-with-resources, or context managers

Assertions & Validation

- Use assertions strictly for **internal invariants**
- Validate all external input, fail fast, and provide clear error messages

Testing

- Use frameworks like **JUnit 5** or **pytest**
- Prefer TDD approach: **Red-Green-Refactor**
- Separate unit / integration / e2e tests with good naming conventions and fixtures

Debugging

- **Concurrency:** protect shared state carefully and avoid deadlocks
- **Distributed systems:** use correlation IDs, retries, circuit breakers, structured logging, and metrics